

Cloud Computing Assignment 1

NAME: Fiza Jabbar

REGISTRATION NO: FA24B1-SE-050

SEMESTER: BSSE-VB

DEPARTMENT: Software Engineering

Task 1: Install Gitea and Push a Repository from the Ubuntu Server

Gitea is a lightweight, self-hosted Git service that provides features similar to GitHub, including repository hosting, user management, authentication, and Git-based collaboration. It can be installed and managed using Docker and Docker Compose.

In this task, Gitea is installed on an Ubuntu Server together with PostgreSQL as its database. Docker containers are used to run the Gitea application and database. After installation, Gitea is accessed through the server's IP address and port 3000.

A separate local Git repository is created on the Ubuntu Server and connected to the Gitea repository using an HTTP remote. A Gitea personal access token is used for authentication when pushing the repository. The token is entered at the password prompt and is not stored in the remote URL.

The purpose of this task is to understand how a self-hosted Git service works and how a local Git repository can be pushed to a private Git hosting server.

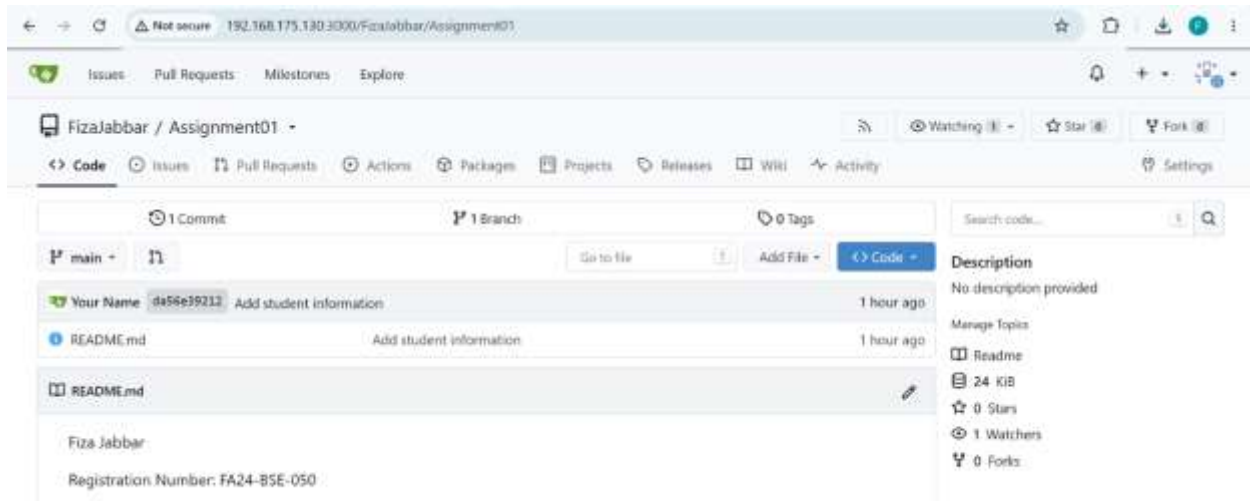
Step 1.1: Install and Run Gitea on the Ubuntu Server

```
fiza@ubuntu:~/Gitea$ docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED      STATUS      PORTS
gitea                gitea/gitea:latest                 "/usr/bin/entrypoint.sh"             gitea      25 minutes ago    Up 25 minutes    22/tcp, 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
gitea_db             postgres:17-alpine                 "docker-entrypoint.sh"               db         25 minutes ago    Up 25 minutes    5432/tcp
fiza@ubuntu:~/Gitea$ curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000/
Gitea HTTP status: 200
fiza@ubuntu:~/Gitea$
```

Step 1.2: Create and Push the Initial Repository

```
fiza@ubuntu:~/Assignment01$ git remote -v
gitea  http://FizaJabbar@127.0.0.1:3000/FizaJabbar/Assignment01.git (fetch)
gitea  http://FizaJabbar@127.0.0.1:3000/FizaJabbar/Assignment01.git (push)
fiza@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://FizaJabbar@127.0.0.1:3000':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 265 bytes | 265.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/FizaJabbar/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'gitea/main'.
```

Step 1.3: Verify the Gitea Repository



Task 2: Push the Same Repository to GitHub

GitHub is a cloud-based platform for hosting Git repositories and supporting software development and collaboration. A Git repository can have multiple remote repositories, allowing the same local repository to be synchronized with different Git hosting services.

In this task, the same local Assignment01 repository used with Gitea is connected to a second remote named github. The repository is then pushed to GitHub using the main branch.

The `git remote -v` command is used to verify both remote repositories. This demonstrates that one local Git repository can communicate with multiple remote repositories.

The purpose of this task is to understand Git remotes and how the same project can be maintained on both a self-hosted Git service and a cloud-based Git service.

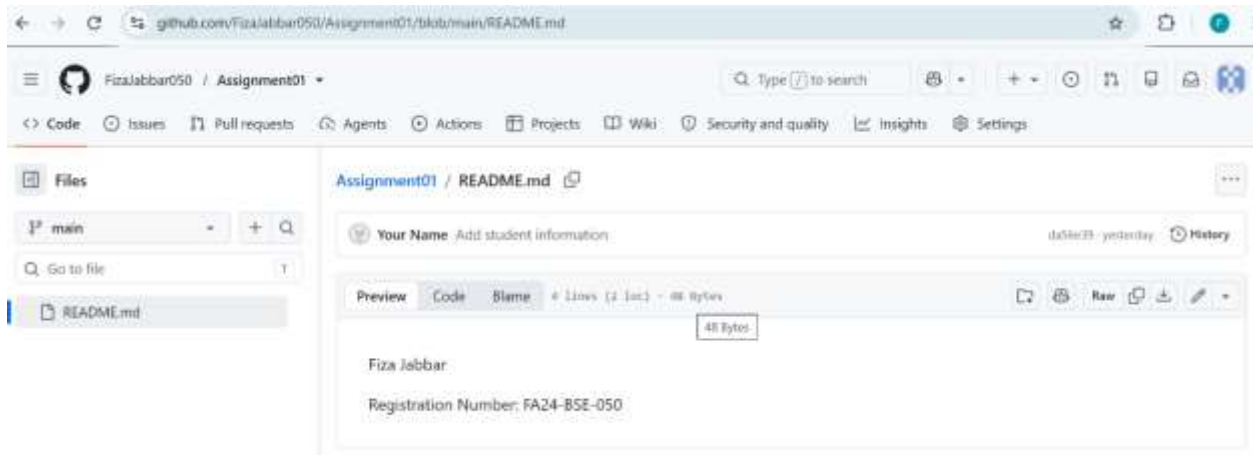
Step 2.1: Create the GitHub Repository and Verify Both Remotes

```
fizajabbar050@ubuntu:~$ cd ~/Assignment01
fizajabbar050@ubuntu:~/Assignment01$ git remote -v
gitea    http://FizaJabbar@127.0.0.1:3000/FizaJabbar/Assignment01.git (fetch)
gitea    http://FizaJabbar@127.0.0.1:3000/FizaJabbar/Assignment01.git (push)
fizajabbar050@ubuntu:~/Assignment01$ git remote add github https://github.com/fizajabbar050/Assignment01.git
fizajabbar050@ubuntu:~/Assignment01$ git remote -v
gitea    http://FizaJabbar@127.0.0.1:3000/FizaJabbar/Assignment01.git (fetch)
gitea    http://FizaJabbar@127.0.0.1:3000/FizaJabbar/Assignment01.git (push)
github   https://github.com/fizajabbar050/Assignment01.git (fetch)
github   https://github.com/fizajabbar050/Assignment01.git (push)
fizajabbar050@ubuntu:~/Assignment01$
```

Step 2.2: Push to GitHub

```
fizaJabbar050@ubuntu:~/Assignment01$ git remote -v
gitea  http://FizaJabbar0127.0.0.1:3000/FizaJabbar/Assignment01.git (fetch)
gitea  http://FizaJabbar0127.0.0.1:3000/FizaJabbar/Assignment01.git (push)
fizaJabbar050@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://FizaJabbar0127.0.0.1:3000':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 265 bytes | 265.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/FizaJabbar/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'gitea/main'.
```

Step 2.4 — Verify on GitHub



Task 3: Track Three Large Files with Git LFS

Git Large File Storage (Git LFS) is an extension of Git designed to handle large files efficiently. Instead of storing the complete contents of large binary files directly in the normal Git repository, Git LFS stores lightweight pointer files in Git while the actual large file contents are managed by LFS storage.

Git LFS is useful for large binary files such as datasets, media files, machine-learning models, and other files that are not efficiently handled by normal Git storage.

In this task, Git LFS is initialized and configured to track files with the .bin extension. Three large binary files are created, each larger than 100 MiB. The files are tracked using Git LFS, committed, and pushed to the GitHub repository.

The `git lfs ls-files --size` command is used to verify that the files are being managed by Git LFS.

The purpose of this task is to understand how Git LFS handles large files and how large binary files can be stored in a GitHub repository without storing their full contents in normal Git history.

Step 3.1: Configure Git LFS

```
fizajabbar050@ubuntu:~/Assignment01$ git lfs version
git-lfs/3.7.1 (GitHub; linux amd64; go 1.25.0)
fizajabbar050@ubuntu:~/Assignment01$ git lfs install
Updated Git hooks.
Git LFS initialized.
fizajabbar050@ubuntu:~/Assignment01$ git lfs track "*.bin"
Tracking "*.bin"
fizajabbar050@ubuntu:~/Assignment01$ cat .gitattributes
*.bin filter=lfs diff=lfs merge=lfs -text
fizajabbar050@ubuntu:~/Assignment01$
```

Step 3.2: Create and Verify Three LFS Files

```
fizajabbar050@ubuntu:~/Assignment01$ truncate -s 101M large-file-1.bin
fizajabbar050@ubuntu:~/Assignment01$ truncate -s 102M large-file-2.bin
fizajabbar050@ubuntu:~/Assignment01$ truncate -s 103M large-file-3.bin
fizajabbar050@ubuntu:~/Assignment01$ ls -lh large-file-*.bin
-rw-rw-r-- 1 fizajabbar050 fizajabbar050 101M Oct  7 00:46 large-file-1.bin
-rw-rw-r-- 1 fizajabbar050 fizajabbar050 102M Oct  7 00:47 large-file-2.bin
-rw-rw-r-- 1 fizajabbar050 fizajabbar050 103M Oct  7 00:48 large-file-3.bin
fizajabbar050@ubuntu:~/Assignment01$ git add .gitattributes large-file-1.bin large-file-2.bin large-file-3.bin
fizajabbar050@ubuntu:~/Assignment01$ git status
On branch main
Your branch is up to date with 'github/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitattributes
    new file:   large-file-1.bin
    new file:   large-file-2.bin
    new file:   large-file-3.bin

fizajabbar050@ubuntu:~/Assignment01$
```

Step 3.3: Commit and Push the LFS Files

[illegible]

Task 4: Create a Portfolio or CV with GitHub Pages

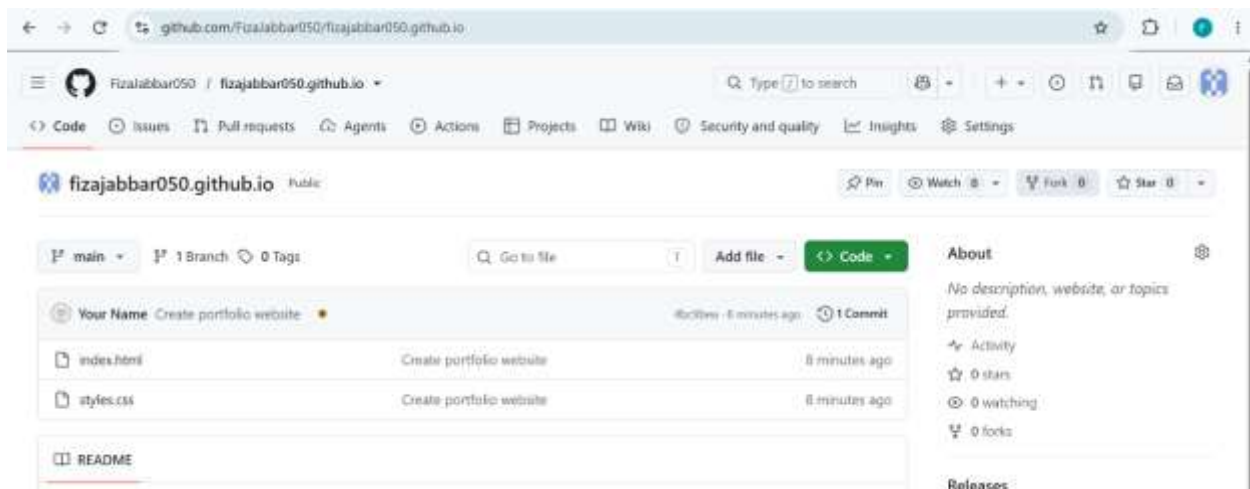
GitHub Pages is a static website hosting service provided by GitHub. It allows HTML, CSS, JavaScript, and other static website files stored in a GitHub repository to be published as a live website.

In this task, a public repository named using the GitHub Pages format <username>.github.io is created. An HTML and CSS-based personal portfolio/CV is added to the repository. The website contains the required sections: About Me, Education, Skills, and Projects.

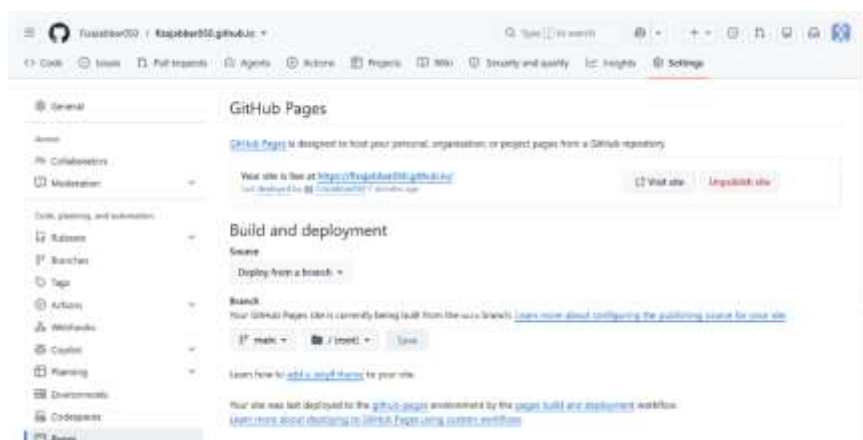
GitHub Pages is configured through the repository's Settings → Pages section. The main branch and the root folder are selected as the deployment source. After successful deployment, the website becomes available through the GitHub Pages URL.

The purpose of this task is to understand how a static website can be hosted and published directly from a GitHub repository.

Step 4.1: Create and Push the Portfolio Repository



Step 4.2: Publish with GitHub Pages



Step 4.3: Verify the Live Portfolio

